Open	Private for human eye (256 bits)	Private and secured by password (256 bits)
Plain data	keccak256(abi.encodePacked(20))	keccak256(abi.encodePacked(20, 0x426526))
20	0xce6d7b5282bd9a3661ae061...	0xf33027072471274d489ff841d4ea9e...

DESIGN PRINCIPLE 7 Ensure the privacy and security of function parameters by secure-hashing the parameters along with a single-use secret password.

Finally, there's the utility function `changeState()`, which can be called only by the beneficiary. The functions `bid()`, `reveal()`, and `auctionEnd()` can be called only when the auction is in the correct phase, as specified by the `validPhase()` modifier defined in listing 5.4.

In the `Reveal` phase, all the (private and secure) valid bids are in by then. The bidder reveals the bid value as well as the secret password. It's okay to reveal the password because it is a one-time, single-use password decided by the bidder. The smart contract function `reveal()` computes the hash of the bid and the secret password, and verifies that it matches that of the blind bid sent earlier. If the hash does match, the contract accepts the bid (the `placeBid()` function) and evaluates it to see whether it is the highest. These checks for verifying the correctness of the blind bid are implemented with `if` statements.

You can add the code in the next listing to the code (listings 5.3 and 5.4) you added earlier to the Remix IDE. Review this code to explore these functions further before moving on to test this smart contract.

Listing 5.5 Functions of the blind auction (BlindAuction.sol)

```
constructor(  ) public {                                    ❶
    beneficiary = msg.sender;
    state = Phase.Bidding;
}

function changeState(Phase x) public onlyBeneficiary {
    if (x < state || x != Phase.Init) revert();
    state = x;
}
```